Ders 7 Çalışma Özeti

Ders 7 Çalışma Özeti

Genel Konular

- Huffman ağacı kurarken yapılan hataların tekrarı
  - Genişletilmiş dizide her toplam sıralamayı bozmayacak yere konur. Örneğin $6 + 7 = 13$, 14'ten sonraya değil 8 ile 14 arasına yazılmalıdır; yanlış yere konan toplam farklı bir ağaç ve farklı (belki prefix bile olmayan) kodlar üretir.
  - Ağaç kurulurken dizi sondan okunur; ilk okunanı sola, ikinciyi sağa koymak şart değildir, ama hangi kural seçildiyse her seviyede aynı uygulanmalıdır. Aksi hâlde kodlar hatalı çıkar.
- Statik, yarı dinamik ve dinamik Huffman (ekrandan, 00:10:00)
  - **Statik:** bir derlemdeki bütün sembollerin kullanım sıklıklarından kodlar bir kez çıkarılır; her doküman için aynı kodlar kullanılır, dokümanın kendi sıklıklarına bakılmaz. Performans en düşüktür.
  - **Yarı dinamik:** her doküman iki aşamada işlenir: önce baştan sona taranıp sıklıklar çıkarılır ve ağaç kurulur, sonra doküman yeniden okunarak kodlanır. Hocanın örneğinde statikte %55 sıkıştırma elde ediliyorsa yarı dinamikte %70'e çıkılabilir.
  - **Dinamik:** hızın önemli olduğu uygulamalarda, özellikle veri A noktasından B noktasına gönderilirken kullanılır. Encoder ve decoder senkron çalışarak kodlama ağacını aynı anda oluşturur; başlangıçta ağaç boştur. Daha karmaşık bir algoritmadır.
- Dinamik Huffman'da escape kodu (ekrandan, 00:10:00 ve 00:20:00)
  - Ağaçta olmayan bir sembol ilk kez geldiğinde kodlanmadan (ASCII olarak) gönderilir, ağaca eklenir ve ona bir kod atanır. Sonraki gelişlerinde sembolün kodu gönderilir ve frekansı bir artırılır; gerekiyorsa ağaç iki tarafta da aynı şekilde yeniden düzenlenir.
  - Decoder'ın, gelenin kodlanmış bir sembol mü yoksa ASCII kodu mu olduğunu bilmesi gerekir. Bu yüzden kodlanmamış sembolden önce bir **escape** kodu gönderilir. Ağaç escape düğümüyle başlar.
  - Escape'in frekansı her zaman 1'dir ve değişmez, çünkü her sembol için yalnız bir kez kullanılır. Ama ağaca yeni sembol eklendikçe escape aşağı seviyelere iner ve kodu değişir.
  - Örnek giriş `a a b c`: önce escape + `a` (ASCII); sonra `a` için kod `0` gönderilir ve frekansı 2 olur; sonra escape + `b`. Bu noktada frekanslar $a = 2$, $b = 1$, escape $= 1$ ve kodlar `a = 0`, `b = 10`, escape `= 11` olur. Sonra escape + `c` gelir ve `c` ağaca eklenir.
  - Bir sembolün kodu dokümanın başında, ortasında ve sonunda farklı olabilir. Bunun sorun olmamasının sebebi header olmamasıdır: iki taraf ağacı aynı anda kurduğu için o anki kodu ağaçtan görür.
- Dinamik ağacın değişimi örneği (ekrandan, 00:26:30 ve 00:29:40)
  - Başlangıç: A (1), B (2), C (2), D (2), E (9). Ara toplamlar $1+2 = 3$, $2+2 = 4$, $3+4 = 7$, $7 + 9 = 16$. Sol dal 0, sağ dal 1 ile `A = 000`, `E = 1`.
  - İkinci A gelince A = 2: toplamlar 4, 4, 8, kök 17; ağaç değişmez.
  - Üçüncü A gelince A = 3: büyük frekanslar sağa yerleştiği için ağaç yeniden kurulur: (D, B) = 4, (C, A) = 5, 9, E = 9, kök 18.
  - Dördüncü A gelince A = 4: sol alt ağaç $4 + 6 = 10$ olur ve E'nin 9'undan büyüktür; bu seviyede 10 ile 9'un yerleri değişir, A, B, C, D ve E'nin kodları birden değişir (kök 19).
  - Beşinci A ile (A = 5) ağaç: E (9) solda; sağda 11 → A (5) ve 6 → C (2) ve 4 → D (2), B (2); kök 20. Bu ağaçta `B = 1111`.
- Dinamik Huffman'da aksayan taraflar (ekrandan, 00:30:00)

  - **Sayıcının taşması:** frekans işaretsiz 16 bitlik bir değişkende tutuluyorsa en fazla $2^{16} - 1 = 65535$ tutulabilir. Taşma olmasın diye kökteki frekans kontrol edilir; en büyük değere ulaşıldığında yaprakların değerlerinin yarısı alınır (integer bölme), ara düğümler ve kök yeniden hesaplanır. Ağacın şekli değişmez. Slayttaki örnek: yapraklar 155, 155, 310, 310 (kök 930) → 77, 77, 155, 155; ara düğümler 154 ve 310, kök 464.
  - **Kod taşması:** kodlar 16 bitlik bir alanda tutuluyorsa ve ağacın seviyesi 16'dan fazlaysa ya alan genişletilir ya da linkli liste kullanılır.
  - 16 seviyeyi geçen ağaç, dokümanda verinin çok dengesiz dağıldığını (çok nadir sembollerin olduğunu) gösterir. Yalnız karakterler kodlanırken beklenmez, ama ikili (bigram) ya da daha uzun semboller kodlanırken ağacın derinliği 16'yı geçebilir; değişken tipi veriyi tanıyarak seçilmelidir.
- Huffman algoritmasının farklı bir versiyonu (ekrandan, 00:40:00)
  - Basit ama daha az etkili bir yöntemdir. Eşit (ya da birbirine yakın) olasılıklı, n elemanlı değişken uzunluktaki kodlar için uygundur; olasılıklar eşit değilse etkili değildir.
  - $n \times 3$ boyutlu bir tablo kullanılır: sembol (name), sıklık (count) ve kod (code). Sıklık değiştikçe satırlar (sembol ve sıklık) yer değiştirir, kod sütunu sabit kalır.
  - Başlangıç: a_1, a_2, a_3, a_4; count'lar 0; kodlar `0`, `10`, `110`, `111`.
  - a_2 gelir: `10` yazılır, count 1 olur; a_2 ile a_1 yer değiştirir. a_4 gelir: `111` yazılır, a_4 ikinci satıra çıkar. Yine a_4 gelir: `10` yazılır, count 2 olur ve a_4 en üste (`0`) çıkar.
  - Bu yöntemde de count değerinde taşma olur; çözüm count değerlerini yarıya bölmektir.
- Canonical Huffman kodlama (ekrandan, 00:46:40 ve 00:50:00)
  - Huffman ağacı full binary tree olmadığı için birçok olası kod oluşmaz ve kod uzunlukları birbirinden çok farklı olur. Kodlar header'da gönderilecekse her kod için en uzun kod kadar yer ayrılması gerekir; örneğin uzunluklar 3, 5 ve 7 bit ise üçüne de 7 bit ayrılır ve 4 ve 2 bit boşa gider. Bu header'ı büyütür.
  - Canonical Huffman geniş alfabeli dosyalarda daha küçük header oluşturur, hızlı kod dönüşümü sağlar ve Huffman'ın atladığı prefix kodların oluşturulmasına izin verir. Hızlı açılma gereken durumlarda (örneğin tıklanınca hemen çalması istenen ses) tercih edilir.
  - Adımlar: (1) her karakter için Huffman kodu geleneksel yöntemle bulunur; (2) kod uzunlukları büyükten küçüğe sıralanır; (3) en uzun uzunluk L_{max} ve bu uzunlukta k_1 kod varsa, L_{max} uzunluğunda 0'dan başlayan k_1 ardışık binary sayı yazılır; (4) bir sonraki uzunluk L_{max-1} ve k_2 kod için bir önceki kodun ilk L_{max-1} biti alınır ve her yeni kod için 1 eklenir; (5) bu, bütün uzunluklar için tekrarlanır.
- Canonical Huffman örneği (ekrandan, 00:56:20)
  - $\{a, b, c, d, e, f, g, h\}$ için Huffman kod uzunlukları $(5, 5, 5, 5, 3, 2, 2, 2)$.
  - $L_{max} = 5$, $k_1 = 4$: `a = 00000`, `b = 00001`, `c = 00010`, `d = 00011`.
  - $L_{max-1} = 3$, $k_2 = 1$: `00011`'in ilk 3 biti `000`, $+1$ → `e = 001`.
  - $L_{max-2} = 2$, $k_3 = 3$: `001`'in ilk 2 biti `00`, $+1$ → `f = 01`, sonra `g = 10`, `h = 11`.
  - Kod uzunlukları değişmez; fark, aynı uzunluktaki kodların ardışık binary sayılar olmasıdır (0, 1, 2, 3). Orijinal Huffman kodlarında böyle bir düzen yoktur.
  - Hoca, ağacın kendisinin header'da gönderilmeye başlanmasından sonra canonical Huffman'ın günümüzde pek kullanılmadığını söyledi.
- MNP5 (Microcom Network Protocol) (ekrandan, 01:20:00 ve 01:30:00)
  - Modem ve fakslar için geliştirilmiş sıkıştırma yöntemlerinden biridir; MNP5 ve MNP7 versiyonları vardır. MNP5 daha basit olduğu için daha çok tercih edilir. Modem ve fakslarda doküman resim olarak gider (her nokta bir piksele ya da pels'e karşılık gelir), bu yüzden RLE iyi sonuç verir.
  - İki aşamalıdır: (1) RLE, (2) adaptive frequency encoding.
  - RLE aşaması: 3 ya da daha fazla tekrar eden byte varsa 3'ü arka arkaya yazılır, dördüncü byte'a tekrar sayısı (fazlası) yazılır. Dokuz tane 52 → `52 52 52 6`.
  - İkinci aşama 256 satırlık ve 2 sütunlu bir tabloyla başlar: her satır 8 bitlik 0-255 arası bir byte'a karşılık gelir; birinci sütun frekanstır (başlangıçta 0), ikinci sütun değişken uzunluklu **token**'dır.
  - Her token 3 bitlik bir header (başlık) ile başlar ve onu kod bilgisi izler. Üç özel durum dışında kodlar 2 tane 1 bitlik (0-1), 4 tane 2 bitlik (0-3), 8 tane 3 bitlik (0-7), 16 tane 4 bitlik (0-15), 32 tane 5 bitlik (0-31), 64 tane 6 bitlik (0-63) ve 128 tane 7 bitlik (0-127)

koddan oluşur. Üç özel kod `000|0`, `000|1` ve `111|11111110`'dur.
  - Okunan byte'ın token'ı çıkışa yazılır ve frekansı 1 artırılır; frekansı büyük olan byte kendi üzerindeki daha kısa token ile gösterilebilsin diye yalnız token'ların yeri değiştirilir. Frekans 8 bitlik bir alanda tutulur; en büyük değere ulaşılırsa bütün frekanslar ikiye bölünür.
  - Örnek `52 52 52 6` (ekrandan, 01:33:30): 52 için tablonun 53. satırındaki token yazılır; her byte'tan sonra tabloda yer değiştirme olabilir ve 52 farklı token'lar alabilir. 6 bir tekrar sayısıdır: tablonun 7. satırındaki token yazılır ama frekansı artırılmaz.
  - Token'lar 4 ile 11 bit arasındadır ve çıkışa 8 bitlik paketler hâlinde yazılır. En sona 11 tane 1'den oluşan **flush token** eklenir (gerekirse 8'e tamamlamak için 1 eklenir). MNP5'in başarısı verinin içeriğine bağlıdır.
  - Tablo güncellemesi iki şekilde yapılır: her artışta bütün tablo yeniden sıralanır ya da tablo içinde pointer'lar kullanılır (ekrandan, 01:40:00). Pointer yönteminde frekans sütunu (F) ile kod sütunu (C) ayrıdır; frekansı artan sembolün pointer'ı daha üstteki (kısa) koda, üzerindekiler bir aşağı kayar.
- MNP7 (ekrandan, 01:50:00)
  - MNP5'ten daha karmaşık ama daha hızlıdır. RLE ve iki boyutlu dinamik Huffman (first order Markov modeline dayalı) birlikte kullanılır.
  - 256 adet 256×2 boyutlu tablodan oluşur. Önceki karakter hangi tablonun, şimdiki (current) karakter o tablodaki hangi satırın kullanılacağını belirler. Her tablonun ilk sütunu 0-255 arası sayılar, ikinci sütunu kullanım sıklıklarıdır (başlangıç 0). Kodlar ayrı bir tabloda tutulur ve asla yer değiştirmez.
  - İkili karakterlerin olasılıkları tek karakterlerinkinden farklıdır (örneğin `a`'nın, `b`'nin ve `ab`'nin sayısı farklıdır). Olasılıklar küçüldükçe birbirine yakın kodlar ve küçük varyans elde edilebilir; bu da iletimde hızlı çözme demektir. Ama kodlaması çok daha zordur; milisaniyeler önemli değilse MNP5 yeterlidir.
- Faks sıkıştırma (ekrandan, 02:20:00 ve 02:23:30)
  - Faksla gönderilen doküman resim olarak gider. ITU standartları: eski T2 (Grup 1) ve T3 (Grup 2); yerlerini T4 (Grup 3, PSTN ağlarında) ve T6 (Grup 4, ISDN'de) aldı. Sıkıştırma oranları 10:1 veya daha yüksektir.
  - Faks makineleri dokümanı satır satır (scan line) tarar ve satırları **pels** (picture elements) adı verilen küçük siyah beyaz noktalara çevirir.
  - Yatay çözünürlük: her zaman 8,05 pels/mm (205 pels/inch, 1″ = 25,4 mm). T4 standardında tarama için önerilen satır genişliği 8,2″.
  - Dikey çözünürlük: standard mode 3,85 scanline/mm, fine mode 7,7 scanline/mm, very fine mode 15,4 scanline/mm.
  - 10″ yüksekliğinde bir sayfa (254 mm) sıkıştırılmadan gönderilirse: standard 978 satır, fine 1956, very fine 3912; satır başına 1664 pels; süreler 170 s (2,82 dk), 339 s (5,65 dk), 678 s (11,3 dk). Süreler, faks iletişiminde sıkıştırmanın ne kadar önemli olduğunu gösterir.
- Grup 3 ve Modified Huffman (ekrandan, 02:28:50 ve 02:30:00)
  - Grup 3 kodu birçok dokümandaki siyah ve beyaz pels'lerin run uzunluklarının analiziyle elde edilmiştir; her run uzunluğuna Huffman ile değişken uzunlukta kod atanır. En sık görülen run'lar 2, 3 ve 4 siyah pels'tir ve bunlara en kısa kodlar atanmıştır.
  - Grup 3, RLE ve Huffman kodlamanın bir bileşimidir. Run değerleri uzun olabileceği için Huffman'da değişiklik yapılmıştır: **MH (Modified Huffman)**.
  - Kodlar ikiye ayrılır: 0-63 run uzunlukları için **termination** kodlar, 64'ün katları (64, 128, 192, …, 2560) için **make-up** kodlar. 1792'den itibaren beyaz ve siyah make-up kodları aynıdır.
  - Bir run kodu ya tek bir termination kodundan ya da bir veya daha fazla make-up kodu ve onu izleyen bir termination kodundan oluşur (ekrandan, 02:37:40): 12 beyaz → `001000`; 76 beyaz (64 + 12) → `11011|001000`; 140 beyaz (128 + 12) → `10010|001000`; 64 siyah (64 + 0) → `0000001111|0000110111`; 2561 siyah (2560 + 1) → `000000011111|010`.
  - Her tarama satırı ayrı kodlanır ve sonuna EOL kodu eklenir. Her satırın soluna bir beyaz pels eklenir; bu, alıcı tarafında belirsizliği kaldırır.
  - Örnek satır `1w3b2w2b7wEOL` → `000111|10|0111|11|1111|EOL`; örnek satır `3w5b5w2bEOL` (aslı 2w5b5w2b) → `1000|0011|1100|11|EOL`.
- Grup 3'te hata ve tek boyutlu kodlama (ekrandan, 02:40:00)
  - Grup 3 hata düzeltmeye sahip değildir ama hataların çoğunu bulabilir. Huffman kodunun doğası gereği tek bir hatalı bit bile alıcıyı senkronizasyondan çıkarır. Bu yüzden

her satır ayrı kodlanır; alıcı hatayı bulursa EOL'a kadar olan bitleri atlar, en fazla bir satır kaybolur.
  - Alıcı satır sayısı kadar EOL göremezse yüksek hata oranı olduğunu kabul eder, işlemi iptal edip göndericiye bildirir. Kodlar 2 ile 12 bit arasında olduğundan 12 bit okuyup geçerli kod bulamayan alıcı hatayı bulur. Sayfa sonunda 6 tane EOL vardır.
  - Bu yönteme **one-dimensional coding** denir: her satır kendi içinde bağımsız işlenir. Text ve siyah-beyaz görüntülerde sıkıştırma oranı yüksektir; kısa run'lardan oluşan, özellikle gray scale görüntülerde başarı düşüktür. Başarıyı artırmak için two-dimensional kodlamaya geçilir.

Hocanın Özellikle Vurguladığı Kısımlar

- Huffman'da tutarlılık
  - Toplamlar sıralamayı bozmayacak yere konmalı; sola/sağa yerleştirme kuralı her adımda aynı uygulanmalı. Küçük bir dikkatsizlik ağacı ve kodları değiştirir, prefix kod bile oluşmayabilir.
- Dinamik Huffman'da küçük varyans
  - Dinamik Huffman kullanılmıyorsa eşit toplamı sola ya da sağa yazmak fark etmez, entropi aynıdır. Dinamik Huffman'da ise varyansı küçük olan yerleşim seçilmelidir. Örneğin 17 bitlik bir kodun ardından 3 bitlik kodlar gelirse alıcı 17 biti çözerken kısa kodlar sırada bekler; büyük buffer gerekir ve hızın önemli olduğu yerde kimse bunu tercih etmez.
- Taşma kontrolü
  - Frekans ve kodların tutulduğu değişkenin tipi (byte, word) önemlidir. Büyük dokümanlarda kökteki sayaç kontrol edilmezse yeni frekanslar kaybolur. Veri tanınarak (ör. bigram kodlanıyorsa) değişken tipi ona göre seçilmelidir.
- Escape kodu
  - Escape'in frekansı değişmez ama kodu değişir; header olmadığı için kodların değişmesi sorun değildir.
- Bilinen algoritmaların türevleri
  - MNP5/MNP7 ve faks sıkıştırmanın özünde RLE ve Huffman vardır. Hoca, yeni algoritmaların temel yöntemlerin (RLE, Huffman, aritmetik kodlama, LZW, PPM) türevleri ya da karışımları olduğunu, verinin içeriğine göre önüne veya arkasına bir işlem eklenerek ortaya çıktığını vurguladı.
- Faksta doküman bir resimdir
  - Faksla gönderilen bilgi karakter değil, siyah-beyaz noktalardır; bu yüzden run uzunlukları üzerinden kodlama yapılır.

Kısa Tekrar Notları

- Statik: derlemden hazır kod; yarı dinamik: iki geçiş; dinamik: tek geçiş, encoder-decoder senkron, header yok.
- Escape: yeni sembolden önce gönderilir, frekansı hep 1, kodu ağaç büyüdükçe değişir.
- `a a b c` örneğinde `b` sonrası kodlar: `a = 0`, `b = 10`, escape = `11`.
- Sayıcı taşması → yaprakları yarıya böl, iç düğümleri yeniden hesapla (930 → 464). Kod taşması → alanı genişlet ya da linkli liste.
- Basit versiyon: $n \times 3$ tablo (name, count, code); satırlar yer değiştirir, kod sütunu sabit.
- Canonical: uzunlukları büyükten küçüğe sırala; L_{max}'ta 0'dan ardışık say; sonraki uzunlukta önceki kodun ilk L bitine 1 ekle.
- $(5, 5, 5, 5, 3, 2, 2, 2)$ → `00000`, `00001`, `00010`, `00011`, `001`, `01`, `10`, `11`.
- MNP5 = RLE (3 kopya + sayı) + adaptive frequency encoding (256 satırlık tablo, 3 bit header + kod, token 4-11 bit, flush token 11 tane 1).
- Tekrar sayısı byte'ı için token yazılır ama frekans artmaz.
- MNP7 = RLE + iki boyutlu dinamik Huffman (önceki karakter → tablo, şimdiki karakter → satır).
- Faks: T4 = Grup 3 (PSTN), T6 = Grup 4 (ISDN); 8,05 pels/mm; 3,85 / 7,7 / 15,4 scanline/mm.
- MH: termination 0-63, make-up 64'ün katları; run = make-up(lar) + termination.
- Her satırın başına 1 beyaz pels, sonuna EOL; sayfa sonunda 6 EOL; kodlar 2-12 bit.

- One-dimensional coding: satırlar bağımsız; gray scale'de başarı düşük.

Detaylı Açıklamalar

- **Dinamik Huffman'ın akışı.** İki taraf da boş ağaçla ve yalnız escape düğümüyle başlar. `a` ilk geldiğinde gönderici escape kodunu ve ardından `a`'nın ASCII kodunu gönderir. Alıcı escape'i görünce arkadan gelenin ASCII kodu olduğunu anlar, `a`'yı çıkışa yazar ve iki taraf da `a`'yı ağacın sol koluna ekler: `a = 0`, frekansı 1. İkinci `a` geldiğinde `0` gönderilir; alıcı bunun escape olmadığını, ağaçta 0 kodlu sembolün `a` olduğunu görür; iki taraf da frekansı 2 yapar. Ağaçta iki sembol olduğu için değişime gerek yoktur. `b` gelince yine escape + ASCII gönderilir; frekanslar $a = 2$, $b = 1$, escape $= 1$ olur ve ağaç yeniden düzenlenir: `a = 0`, `b = 10`, escape = `11`. `c` gelince escape + ASCII gönderilir ve escape bir seviye daha aşağı iner. İşlem giriş bitene kadar sürer. Bir sembolün kodu belgenin farklı yerlerinde 3, 5 ya da 4 bit olabilir; bu algoritmayı hızlı ama karmaşık yapar.
- **Ağacın yeniden kurulması.** Dinamik örnekte A'nın frekansı arttıkça hoca her seferinde "sıralasaydım yine bu ağaç çıkardı" diye kontrol etti: A = 3 iken frekanslar 2, 2, 2, 3, 9'dur ve Huffman bunu (2, 2) = 4, (2, 3) = 5, 9, 9, 18 diye birleştirir. A = 4 iken (2, 2) = 4, (2, 4) = 6, $4 + 6 = 10$ ve 10 > 9 olduğundan alt ağaç E'nin sağına geçer. Büyük frekanslı düğümler sağa yerleştirilir.
  - Ders sonrası not: Slayttaki şeklin (00:26:30) son ağacında A'nın frekansı (5) yazılı görünür; oysa ara düğüm 10 ve kök 19 değerleri A = 4 ile tutarlıdır. Bir sonraki şekil (kök 20) A = 5 durumudur.
- **Taşmada yarıya bölme.** Slayttaki örnekte yapraklar 155, 155, 310, 310 ve kök 930'dur. Yapraklar integer bölmeyle yarıya inince 77, 77, 155, 155 olur; ara düğümler yeniden toplanarak 154 ve 310, kök 464 bulunur. Kökün doğrudan yarısı ($930/2 = 465$) alınsaydı yanlış olurdu; bu yüzden ara düğümler yeniden hesaplanır (python ile kontrol edildi).
  - Ders sonrası not: Hoca 00:41:03'te word için sınırı "32 bin bilmem ne" diye söyledi; slayttaki işaretsiz 16 bitlik değişken için sınır $2^{16} - 1 = 65535$'tir (32767, işaretli 16 bit sınırıdır).
- **Canonical örneğinde düzeltme.**
  - Ders sonrası not: Hoca 00:52:47'de dördüncü 5 bitlik kodu (`00011`) "E'nin kodu" diye söyledi; slayttaki örneğe göre bu `d`'nin kodudur. `e`, 3 bitlik `001` kodunu alır. Bulunan kodlar prefix'tir ve Kraft toplamı $4 \cdot 2^{-5} + 2^{-3} + 3 \cdot 2^{-2} = 1$'dir (python ile kontrol edildi).
- **MNP5 token tablosu.** Slayttaki tablodan okunan token'lar: 0 → `000|0`, 1 → `000|1`, 2 → `001|0`, 3 → `001|1`, 4 → `010|00`, …, 7 → `010|11`, 8 → `011|000`, …, 15 → `011|111`, 32 → `101|00000`, 247 → `111|1110111`, 254 → `111|1111110`, 255 → `111|11111110`. Bu kurala göre 52'nin ilk token'ı `101|10100`, 6'nınki `010|10` olur (ara adım eklendi; derste token değerleri okunmadı).
  - Ders sonrası not: Hoca 01:23:56'da 3 bitlik header'ların gray kodlamaya göre konduğunu söyledi. Slayttaki header dizisi (000, 001, 010, 011, …) gray kodu değildir (3 bitlik gray dizisi 000, 001, 011, 010, 110, 111, 101, 100'dür). Ayrıca slaytta 16-31 için header `111` görünür; 128-255 de `111` ile başladığı için `111|0000` kodu `111|0000000`'ın öneki olur ve prefix özelliği bozulur. Büyük olasılıkla header'lar sırayla 000-111'dir ve 16-31 için doğru header `100`'dür.
  - Ders sonrası not: Hoca 01:37:41'de token'ların "ya 4 bit ya da 11 bit" olduğunu söyledi; slayta göre token'lar 4 ile 11 bit arasında değer alır.
- **Gray kodlama.** Hoca header'ı açıklarken gray kodunu anlattı (ekrandan, 01:25:45): binary kodlamada bir koddan sonrakine geçerken birden fazla bit değişebilir (ör. `011` → `100`'de üç bit). Gray kodlamada her geçişte yalnız bir bit değişir: `000`, `001`, `011`, `010`, …
- **MNP5 pointer'lı güncelleme.** Başlangıçta bütün frekanslar 0'dır ve her satır karşısındaki kodu gösterir. `a` geldiğinde `a`'nın gösterdiği kod çıkışa yazılır, frekansı 1 olur; üzerindekilerin frekansı 0 olduğu için `a`'nın pointer'ı adım adım en üstteki koda kadar çıkar,

aradakiler birer aşağı iner. İkinci `a` geldiğinde pointer izlenerek üstteki kod yazılır ve frekans 2 olur; üstünde daha küçük frekanslı değer olmadığı için yerinde kalır. Sonra gelen semboller de frekansları arttıkça `a`'nın altına yerleşir.

- **MNP7'nin sezgisi.** Hoca bir metin üzerinde `a`, `b` ve `ab` sayılarını saydı (a yaklaşık 28, b 5, ab 3). Tek karakter olasılıklarıyla ikili karakter olasılıkları farklıdır; `ab` üç kez geçiyorsa `ba` aynı sayıda geçmeyebilir. Bu nedenle bir karakteri önceki karaktere göre kodlamak farklı ve daha yakın olasılıklar, dolayısıyla daha küçük varyanslı kodlar verebilir.
- **Faks süre tablosu.** 10″ = 254 mm; $254\times3{,}85 \approx 978$, $254\times7{,}7 \approx 1956$, $254\times15{,}4 \approx 3912$ satır. Satır başına 1664 pels ile sayfa başına pels sayıları 1,627M, 3,255M ve 6,510M'dir.
  - Ders sonrası not: Slayttaki tabloda standard mode için sayfa başına pels 1.670M yazar; $978\times1664 = 1.627.392$ olduğundan büyük olasılıkla doğrusu 1,627M'dir (python ile kontrol edildi).
  - Ders sonrası not: Hoca 02:24:21'de tablodaki 678 saniyeyi sıkıştırılmış gönderim süresi gibi anlattı. Slayttaki soru sıkıştırma yapılmadığı durumu sorar; 678 s ile 11,3 dk aynı sürenin iki birimidir ve üç süre de sıkıştırmasız gönderim süreleridir.
  - Ders sonrası not: Slaytta "8,5″ genişliğinde taranan her satır 1.738 pels" yazar; $8{,}5\times205 \approx 1742$'dir. Büyük olasılıkla kastedilen, T4'te satır başına standart 1728 pels'tir.
- **Make-up kodunun sırası.**
  - Ders sonrası not: Hoca 02:33:59'da 76 beyaz için önce termination, sonra make-up kodunun yazıldığını söyledi; slayttaki örneğe göre önce make-up (`11011`, 64), sonra termination (`001000`, 12) yazılır.
  - Ders sonrası not: Hoca 02:27:06'da 2-7 arası beyaz run'ların az görüldüğü için uzun kod aldığını söyledi. Tabloda bu run'ların kodları 4 bittir (`0111`, `1000`, `1011`, `1100`, `1110`, `1111`); büyük olasılıkla slayttaki "daha uzun", siyah 2-4 run'larının 2-3 bitlik kodlarına göre biraz daha uzun demektir.
- **Satır kodlama örnekleri.** Birinci örnekte satır 3 siyah, 2 beyaz, 2 siyah, 7 beyazdır (14 pels); başına 1 beyaz eklenir: `000111` (1 beyaz), `10` (3 siyah), `0111` (2 beyaz), `11` (2 siyah), 1111 (7 beyaz), sonra EOL. İkinci örnekte satır 2 beyaz, 5 siyah, 5 beyaz, 2 siyahtır; başa bir beyaz eklenince 3 beyaz olur: `1000`, `0011`, `1100`, `11`, EOL. Aradaki | ayıraçları yalnız tablodan bakmayı kolaylaştırmak içindir; gerçek çıkışta yoktur. Bütün kodlar slayttaki termination tablosuyla karşılaştırıldı ve tuttu.
  - Ders sonrası not: Slayttaki EOL kodu `0000000001` biçiminde okunuyor; ITU T4 standardında EOL büyük olasılıkla 11 sıfır ve bir 1'den oluşan 12 bitlik `000000000001` kodudur (kodların 2-12 bit arasında olmasıyla da uyumludur).
- **Hata durumları.** Bir satırdaki bit bozulup başka bir geçerli koda dönüşürse yalnız o satır hatalı çizilir; bu, okunabilirliği çok bozmaz. Bozulan bit dizisi tabloda yoksa alıcı 12 bite kadar okuyup kod bulamayınca o satırı hatalı sayar ve EOL'dan devam eder. EOL'u göremezse hata sonraki satırlara yayılır; satır sayısı kadar EOL yoksa belge yeniden istenir.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.